MOSIP DECODE 2026 · PROBLEM STATEMENT 02 · HIGH COMPLEXITY

MOSIP Inji Conformance Laboratory

A zero-trust interoperability playground that runs the full Issue → Hold → Present → Verify lifecycle, inspects every protocol byte — then attacks its own verifier to prove it fails closed.

51/51	4	8+10	920 MB
automated checks passing	credential formats end-to-end	adversarial attack vectors	hard Docker RAM ceiling

THE PROBLEM

A valid signature is not a valid credential

Cross-ecosystem credential exchange fails in the field, and nobody can see why. Wallets crash on draft mismatches; verifiers silently accept what they should reject.

Draft 13 ⇄ 14

Draft drift

OpenID4VCI Draft 13 wallets break against Draft 14 verifiers — singular proof vs plural proofs DTO.

Type confusion

Silent acceptance

INJIVER-1131: a LifeInsuranceCredential with a valid signature passes a national-ID presentation request.

RFC 9449 / 9901

Binding gaps

DPoP ath mismatches and key-binding JWTs that never verify sd_hash or audience let replayed proofs through.

Black box

No visibility

Raw protocol payloads are invisible — judges, integrators and QA cannot inspect a single hop.

Every failure above is reproducible in this laboratory — and every defense is enforced by a test that fails loudly if it regresses.

THE SOLUTION

A conformance playground, not another mock

ISSUER

- Mock Inji Certify (Draft 13)
- Live Collab sandbox probe
- Draft 14 transpilation mode

→

WALLET

- Headless synthetic agent
- Real ephemeral P-256 keys
- DPoP + kb+jwt proof engine

→

VERIFIER

- Mock Inji Verify (Draft 21-23)
- DIF PE 2.0 + DCQL queries
- Fail-closed trust registry

Execute any permutation of the 3 lanes across 4 credential formats with one click

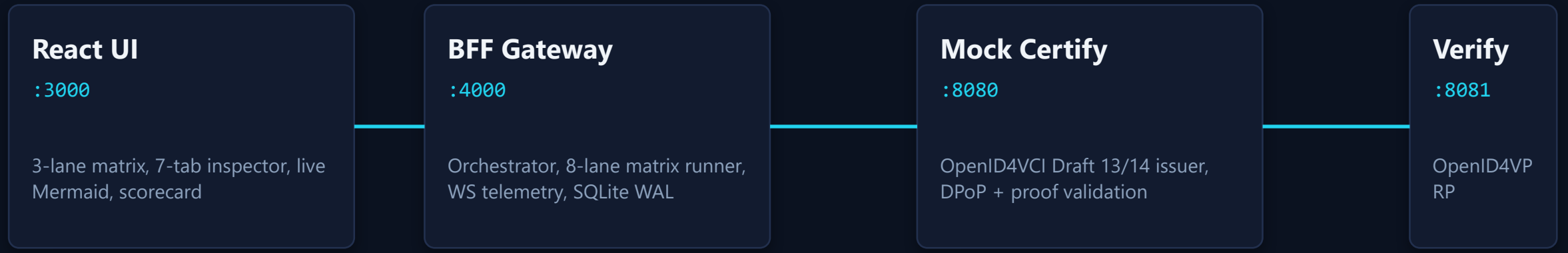
Inspect every raw payload — decoded JWTs, disclosures, digests, MSO valueDigests, live WS telemetry

Attack 8 chaos vectors + 10 adversarial probes must each be rejected with its exact RFC 7807 code

Report honest scorecard, SQLite history, and upstream RFC 7807 bug exports ready to file

ARCHITECTURE

Four services, one command, real cryptography




```
$ docker compose up --build   →  4 healthchecked services, ordered startup, 920 MB hard ceiling  ·  or zero-Docker: node scripts/start_local.js
```

- RFC 7515 · JWS P1363 64-byte r||s, low-S
- RFC 9449 · DPoP ath, jti, htm/htu, cnf.jkt
- RFC 9901 · SD-JWT sd_hash key binding
- RFC 7638 · JWK thumbprints
- ISO 18013-5 · COSE_Sign1 + valueDigests

No stubbed verification.
No string-matching shortcuts.
Every signature is checked with real crypto.

PROTOCOL INSPECTION

Every byte, on screen, in real time

`vc+sd-jwt`	IETF SD-JWT VC — 5 salted disclosures + 3 decoy digests, RFC 9901 kb+jwt
`dc+sd-jwt`	Digital Credentials format — same engine, DC-style route
`ldp_vc`	W3C VC JSON-LD — Ed25519Signature2020 proofs
`mso_mdoc`	ISO 18013-5 — CBOR Tag 24, COSE_Sign1, per-attribute MSO digests

Token anatomy — color-coded header / claims / signature / disclosures / kb-jwt — with SHA-256 MATCH badges computed live in your browser against the issuer _sd array

Live telemetry — WebSocket stream of every hop with millisecond latency; origin-guarded (CSWSH 403); PII redacted

Mermaid sequence — diagram generated from the actual selected topology — re-renders when you pick an attack vector

Privacy slider — toggle disclosed claims; QR density + privacy score react; the verifier enforces what it requested (DIF PE 2.0)

ADVERSARIAL ENGINE

8 chaos vectors — every attack must fail closed

Vector	Attack	Required rejection
VEC-01	Issuer signature bit-flip (real 1-bit crypto corruption)	ERR_INVALID_ES256_SIGNATURE
VEC-02	kb+jwt iat backdated beyond ±300s skew	ERR_KB_JWT_IAT_OUTSIDE_SKEW_WINDOW
VEC-03	Cut-and-paste sd_hash desync	ERR_KB_SD_HASH_MISMATCH
VEC-04	Unbound disclosure injection (digest ∉ issuer _sd)	ERR_UNBOUND_DISCLOSURE_DIGEST
VEC-05	Expired credential presentation	ERR_CREDENTIAL_EXPIRED
VEC-06	Recursive _sd nesting loop	ERR_RECURSIVE_SD_NESTING_LIMIT_EXCEEDED
VEC-07	INJIVER-1131 credential type confusion	ERR_INJIVER_1131_WRONG_CREDENTIAL_TYPE
VEC-08	ASN.1 DER signature where P1363 is mandated	ERR_NON_P1363_SIGNATURE_LENGTH

+10

independent adversarial probes driven with attacker-held keys

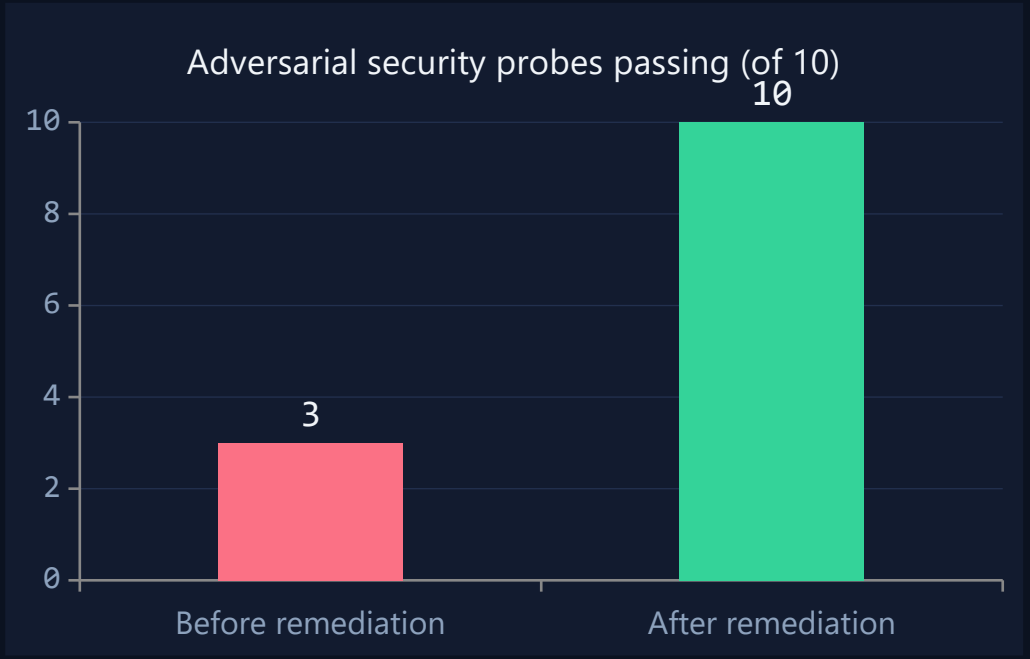
forged tokens · stripped sd_hash · foreign audience · withheld claims · byte-flipped Ed25519 proofs · replayed nonces · rogue issuers · DPoP htm attacks

A vector passes only when the verifier returns HTTP 400 with its exact error code — any other outcome fails the suite.

ENGINEERING INTEGRITY

We did not trust our own first audit

An internal report once claimed "100% everywhere." We discarded it, re-derived every claim from live execution, and attacked the system with an auditor-written probe suite using attacker-held keys.



7 live-exploited vulnerabilities found
forged access tokens issued credentials; kb+jwt without sd_hash or with a foreign audience verified; zero-claim presentations passed; PII persisted unredacted

Every defect fixed & locked
each fix is pinned by the exact probe that originally broke it — the suite is now a permanent regression gate in CI

Honesty applied to the UI itself
the scorecard no longer hardcodes "100%"; anatomy badges compute real SHA-256 digests; everything simulated is labeled "Simulated"

Full evidence ledger ships in the repo
AUDIT_REPORT.md — defect register with IDs, root causes, fixes and tests; docs/verification-matrix.md — requirement-to-test traceability

"We would rather show you an honest defect register than a polished claim."

JUDGE MODE & EVIDENCE

One click, 51 checks, zero trust required

Run Judge Conformance Matrix

8 concurrent lanes — vc+sd-jwt · dc+sd-jwt · ldp_vc · mso_mdoc · Draft-14 transpiled · live Collab probe · inji-verify lane — scored in ~0.4s with an honest scorecard and exportable report.

```
$ node scripts/test_e2e_local.js
FINAL VERIFICATION RESULTS: 26/26 CHECKS PASSED (100%)
$ node scripts/test_security_regression.js
SECURITY REGRESSION RESULTS: 10/10 PASSED
```

26/26 **Chain of Verification —** DPoP · sd_hash · P1363 · 8 chaos vectors · replay · mdoc

10/10 **Adversarial Security Suite —** attacker-held keys — was 3/10 before remediation

4/4 **mdoc Tamper & Replay —** MSO digest mismatch · digestID · nonce replay

8/8 **CI Interop Matrix —** fail-closed CLI + GitHub Action on every push

PRODUCTION READINESS

Boring where it counts

920 MB

hard Docker RAM ceiling across 4 containers (budget: 1,500 MB)

< 100 ms

inspector updates over WebSocket per protocol hop

0

npm audit vulnerabilities across all services (HIGH body-parser patched)

1

command to run: docker compose up --build — offline, no config

Apache-2.0

OSI-approved license; zero proprietary or paid APIs

~15 MB

idle V8 heap per service — runs on a laptop

Healthchecked containers · ordered startup · SQLite WAL persistence · CI workflow on push/PR/manual · fail-closed test exits on every suite

Proven, not promised.

Real cryptography. Four credential formats. Eighteen attack vectors, each rejected with its exact error code. An honest audit that found and fixed its own vulnerabilities before you could. One command, 920 MB, Apache-2.0.

VERIFY EVERYTHING YOURSELF

```
$ node scripts/start_local.js
$ node scripts/test_e2e_local.js
$ node scripts/test_security_regression.js
$ bash scripts/cli_interop_runner.sh
```

FULL DOCUMENTATION IN THE REPO

`README.md`
setup, architecture, compliance matrix, audit method

`AUDIT_REPORT.md`
defect register, evidence ledger, release assessment

`docs/verification-matrix.md`
requirement-to-test traceability, honest statuses

`JUDGE_TESTING_GUIDE.md`
acceptance checklist — verify every claim yourself